ARO by Hallucination

A language built by a conversation, and how to continue the conversation yourself

ARO Language Project

April 2026

Contents

ARO by Hallucination **3**

Chapter 1: Genesis **4**
1.1 The First Conversation . . . 4
1.2 Letting the Machine Write . . . 4
1.3 The First Full Application . . . 5
1.4 The Problem With General Models . . . 5
1.5 Why "Hallucination" . . . 5

Chapter 2: Training the Local **6**
2.1 Why Small . . . 6
2.2 What the Pipeline Does . . . 6
2.3 What the Model Was Taught . . . 7
2.4 What the Model Was Not Taught . . . 8
2.5 Running the Model . . . 8
2.6 A Note on Iteration . . . 8

Chapter 3: The Commands **9**
3.1 The First Invocation . . . 9
3.2 Backends . . . 9
3.3 One-shot vs. REPL . . . 9
3.4 The Slash Commands . . . 10
3.5 Flags . . . 11
3.6 The Context File . . . 11

Chapter 4: Habits **13**
4.1 Context Is Small . . . 13
4.2 Let It Read the Project . . . 13
4.3 Approve Shell Commands Like You Mean It . . . 13
4.4 Parse Before You Write . . . 14
4.5 Keep the Index Fresh . . . 14
4.6 Read the Proposals Before You Argue . . . 14
4.7 Commit Messages Are Not the Model's Job . . . 14
4.8 Precise Prompts Are Better . . . 15

Chapter 5: Automation Everyone Understands **16**
5.1 The Case for Readable Automation . . . 16
5.2 The First Useful Script: A Nightly Report . . . 16
5.3 CI-style Invocations . . . 17
5.4 Scheduled Jobs . . . 18
5.5 Talking to Other MCP Servers . . . 18
5.6 A Principle . . . 18

Chapter 6: Reclaiming the Local **20**
6.1 The Drift to the Cloud . . . 20
6.2 What Changed . . . 20
6.3 What `aro ask` Is For . . . 20
6.4 The Recursive Game . . . 21
6.5 What You Can Do Now . . . 21
6.6 Where This Ends . . . 21

Chapter 7: Tool Calling **23**
7.1 The Eighteen Tools . . . 23
7.2 How the Loop Works . . . 23
7.3 The Shape of a Tool Call . . . 24
7.4 Approval and Trust . . . 24
7.5 Chaining: Read, Edit, Check . . . 25
7.6 Creating a Project from Scratch . . . 25
7.7 The Limits of Tools . . . 26
7.8 Tips for Better Tool Usage . . . 26

Chapter 8: Tips and Tricks **27**
8.1 Writing Good Prompts . . . 27
8.2 Common Mistakes the Model Makes . . . 27
8.3 Using /fix vs. Manual Editing . . . 28
8.4 Building Incrementally . . . 28
8.5 Using .context Effectively . . . 29
8.6 Combining aro ask with the CLI . . . 29
8.7 Performance Tips . . . 29

Chapter 9: Worked Examples **31**
9.1 Example 1: A User Service from Scratch . . . 31
9.2 Example 2: A File Watcher . . . 34
9.3 Example 3: A Plugin from Scratch . . . 35
9.4 What the Examples Show . . . 37

Appendix A: The Training Pipeline **38**
A.1 Prerequisites . . . 38
A.2 The Notebooks . . . 38
A.3 The Model Lifecycle . . . 40
A.4 Iterative Improvement . . . 40
A.5 Key Configuration . . . 40

ARO by Hallucination

A language built by a conversation, and how to continue the conversation yourself.

This is not a reference manual. The Language Guide already does that job, and it does it well. This is something different: a short book about how ARO came into the world and what you can do now that it has. It is the story of a language that was designed by talking with a machine, written by that machine, and then used to train *another* machine — one small enough to live on your laptop, and stubborn enough to keep talking back.

The cover word — *hallucination* — is what critics call it when an LLM makes something up. Most of ARO was made up first, in exactly that sense, during a conversation. Then it was checked, edited, pushed back on, compiled, broken, fixed, and eventually shipped. Hallucination is not the opposite of rigor. It is its starting condition.

If you picked this up looking for the definitive list of actions or the specification of the type system, put it back down and pick up the Language Guide. If you picked it up because you want to understand *how* ARO was built, or because you want to build on it the same way — yes, sitting at a laptop, talking to a local model — keep reading.

— The ARO team, April 2026

Chapter 1: Genesis

> "There are two ways to write a language. One is to know exactly what you want. The other is to ask until you find out."

1.1 The First Conversation

ARO did not begin with a grammar. It began with a complaint. The complaint went roughly like this: every piece of business software, no matter the domain, ended up expressing the same three things — what happens, what we have afterwards, and what it applies to. Every form, every API, every back-office job. And yet every framework, every language, every architecture pattern found a different way to bury those three things under ceremony.

That complaint was typed into a chat window. The reply — from a general-purpose language model — was a sketch. Not a specification. A sketch. Three words. *Action, result, object.* The shape of every sentence in English business prose. "Retrieve the user from the repository." "Compute the total from the line items." "Publish the invoice to the queue." The sketch fit every example the conversation threw at it.

That was the hallucination. A useful one. The next hour of conversation was spent asking the model to break it — to find a case where the shape didn't fit. It failed. Or rather, it found cases where you could argue about prepositions, but the shape held. At the end of that hour, the first line of ARO was typed:

```
Retrieve the <user> from the <user-repository> where id = <id>.
```

It has barely changed since.

1.2 Letting the Machine Write

Once the shape was fixed, the question became: who writes the first thousand lines? The honest answer was "nobody, yet". Nobody knew ARO — it didn't exist outside the conversation window. The parser, the lexer, the action registry, the event bus, the OpenAPI integration, the plugin loader, the LLVM backend — every single one of those components was written in the same way. A human asked for a component. A model produced a draft. The human compiled it, ran the tests, pointed out the failures, and asked for a fix. The model fixed it. Repeat until it ran.

This is not a triumphal story. The drafts were wrong about half the time. The model hallucinated Swift APIs that did not exist. It got the concurrency model subtly wrong in ways that only showed up under load. It invented test frameworks. Every one of those wrong drafts was fixed — not by magic, not by "prompting harder", but by reading the code and understanding what it was trying to do, and then correcting it by hand or by going back to the model with a tighter question.

What made it work was not that the model was smart. It was that the language being built — ARO — was small. The grammar fits on one page. The action roles are five categories. The concurrency model follows one rule. When a component was wrong, the smallness of the language made the wrong part easy to see.

The lesson was not "you can ship software without reading code". The lesson was: you can move a lot faster than you think you can, *if you already know what shape you want.*

1.3 The First Full Application

The first ARO program that ran end-to-end was a user service. An OpenAPI contract with `listUsers`, `createUser`, `getUser`. Six feature sets, one `Application-Start`, a repository, a handful of events. It compiled, it ran, it served HTTP traffic. The conversation window that produced it is still in a text file somewhere — thousands of turns, mostly fixing things the model got almost right.

The important thing about that first application is not that it worked. It is that it was *readable*. Someone who had never seen ARO before could open `listUsers.aro` and understand what it did, because every statement was an English sentence. That was always the point: ARO is a language for business logic, written in the shape that business people already speak in. The fact that a language model — trained on business prose — could guess at ARO was not a happy accident. It was the whole design.

1.4 The Problem With General Models

And yet — ARO has a problem with general models. The problem has two parts.

The first part is vocabulary. There are five action roles, ten prepositions, and fifty-four built-in action verbs. The model that wrote the first version of ARO does not know any of them. It has to be reminded, every time, which verbs exist and which prepositions they take. It tends to invent new ones when it runs out of patience. Invented verbs do not parse. Feature sets with invented verbs fail silently in review and loudly in production.

The second part is privacy. Large general models live in a cloud. They see your source, your questions, and — if you're not careful — the production data you paste in when you're trying to debug something. That's a problem for teams that cannot ship their business logic to a vendor for analysis. It's a problem for people who simply do not want a transcript of every engineering decision leaving their machine.

The solution to both problems is obvious once you say it out loud: train a small model on ARO, and run it locally. The obvious solution is what this book is about.

1.5 Why "Hallucination"

The title of the book is a joke with a sharp edge. *Hallucination* is what critics call it when an LLM makes something up. It is also, quite accurately, how ARO was born — a chain of well-placed hallucinations, each one checked and kept or thrown away. The derogatory use of the word "hallucination" assumes that making things up is bad. That is wrong. Making things up is how design works. What matters is what happens *after* you make them up.

The rest of this book is about what happens after. Chapter 2 tells the story of training the local model. Chapter 3 teaches the commands. Chapter 4 is about good habits. Chapter 5 is about what you can do with `aro ask` that you cannot do with any other tool. Chapter 6 is about why all of this matters — why the local machine is worth claiming back. Chapter 7 explains how the tool-call loop works. Chapter 8 collects practical tips. And Chapter 9 walks through complete examples from scratch.

Chapter 2: Training the Local

> "The question is not whether a model can learn ARO. The question is how small it can be and still do it."

2.1 Why Small

The fine-tuned model that ships with `aro ask` is not large. By the standards of 2026, it is small — a 4-bit quantised dense model with 8 billion parameters, small enough to run comfortably on a laptop with 8 GB of memory or a mid-range GPU. This was not a compromise. It was the point.

Behind the scenes, a much larger teacher model — a 30-billion-parameter mixture-of-experts architecture — is fine-tuned on ARO and then used to *distil* its knowledge into the small model that ships to users. You never see the teacher. You get the student: fast, compact, and focused.

A large model trained on ARO would be impressive. It would also be useless, because nobody would be able to run it. The decision to build a local assistant meant the decision to build a small one. Everything about the training pipeline was designed around that constraint. The goal was never "state of the art on general code". The goal was "good enough on ARO to make a working engineer faster".

2.2 What the Pipeline Does

The training pipeline lives in the sibling project `ARO-Train`. It is a sequence of Jupyter notebooks — numbered `00` through `24` — that collect data, shape it into pairs, fine-tune a teacher model, distil it into a student, and package the result. You do not need to run it to use `aro ask`. You only need to run it if you want to train your own variant.

In prose, the pipeline does the following:

1. **Collects a corpus** from every authoritative source in the ARO project — proposals, the Book, the wiki, every example application, the commit history of the language itself, and the README. This is the only place where the training process touches the public internet; after this step, everything happens locally.
2. **Extracts knowledge** from that corpus into a structured file — the grammar, the list of actions with their verbs and prepositions, the canonical syntax, and the error philosophy.
3. **Generates instruction/response pairs** from the knowledge file, from the books, from the git history of the language, and from comments left in existing ARO applications. Every example and application ships with a `plan.md` — a detailed implementation plan — and the pipeline generates ten paraphrases of each plan, all paired with the same verified output. This teaches the model to recognise many different ways of asking for the same thing. The same ten-variant strategy is applied to every training pair in the pipeline: application prompts, code-to-comment explanations, and example-based code generation all produce ten instruction variants per ground-truth response.
4. **Warm-starts a fine-tune** on a general base model — by default a 4-bit Qwen coder — using the pairs.
5. **Runs a preference optimisation pass (DPO)** using pairs of good and bad ARO answers, so the model learns which of two superficially correct responses is the one a human would actually want.

6. **Evaluates** the fine-tune against a fixed set of probe prompts, checks the output with `aro check`, and reports a syntax pass rate.
7. **Distils** the large 30B teacher into a compact 8B student model, so users get fast inference without needing 16 GB of memory.
8. **Packages** the resulting weights for distribution and uploads both the teacher (for future retraining) and the student (for users) to HuggingFace.

The entire pipeline was written by the same method as the rest of ARO: a person asking a general model what each step should look like, reading the draft, running it, fixing it, and asking for the next step. The training script that produced `aro-coder-4bit` was itself the product of a long conversation with a model that did not know what ARO was, and that needed to be told.

The irony is load-bearing. A model that did not know ARO wrote the code that trained a model that does.

2.3 What the Model Was Taught

The fine-tuned model learned five things, in roughly decreasing order of how much pipeline time was spent on each:

The shape of every statement. Every line of ARO is *verb the <result> preposition [the] <object>*. The model saw thousands of statements in that shape and learned to produce new ones in the same shape without dropping the angle brackets, without inventing prepositions, and without drifting into Python or Swift syntax halfway through.

The five action roles. REQUEST actions (Extract, Retrieve, Read, Fetch) pull data from outside. OWN actions (Compute, Validate, Compare, Create, Transform) work inside the process. RESPONSE actions (Return, Send, Log, Store) hand data back or persist it. EXPORT actions (Emit, Schedule) push data to external consumers. SERVER actions (Start, Stop, Connect, Listen) manage services and infrastructure. The model learned which verbs live in which category, and which prepositions each verb actually takes.

The feature set shape. A feature set is `(Name: Business Activity) { statements }`. Feature sets are triggered by events — HTTP requests, domain events, repository changes — not called directly. The model learned to write complete, runnable feature sets given only the trigger and a one-line description of what should happen.

The error philosophy. ARO feature sets contain only the happy case. Errors are handled by the runtime. The model learned to resist adding defensive `when` guards, try/catch constructs, or error returns — because none of those exist in ARO.

What does not exist. A model trained only on correct examples will cheerfully invent plausible-sounding actions — Tail, Scan, Update, Query — that are not part of the language. The training pipeline includes explicit correction data: pairs where the user asks for a non-existent action and the model explains which real action to use instead. The validation pass rejects any generated sample that uses a verb not in the canonical action list.

It was also taught, almost as an afterthought, how to wrap its output in markdown fences and how to cite the proposal number when asked about a design decision. These are small things. They compound into the difference between a useful assistant and an exasperating one.

2.4 What the Model Was Not Taught

The pipeline deliberately did not teach the model several things.

It was not taught to hold your hand. There are no long prefaces, no "I'll be happy to help with your request" boilerplate, no summary at the end of every reply. The training data was stripped of these. You get the answer, or you get an ARO code block, and then the model stops talking.

It was not taught to pretend it knows things it does not. When asked about features that do not exist in ARO, the fine-tune will say so and point you at the closest proposal. The alternative — confidently inventing features — was exactly the failure mode that killed general models on ARO. The DPO pass pushed hard against that mode.

It was not taught general web knowledge. The corpus is *just* the ARO project. Ask the fine-tune about Kubernetes and it will redirect you to its general base model's knowledge, which is stale and thin on ARO terms. Ask it about the EventBus and it will quote you the relevant proposal.

2.5 Running the Model

The distilled student is published as `ARO-Lang/aro-coder-4bit` on Hugging Face. It is loaded automatically the first time you run `aro ask`. You do not need to run the training pipeline to use the model — the first `aro ask` invocation will offer to download the weights to `~/.cache/aro/ask/`.

The interactive loop inside the training pipeline (notebook `24_chat.ipynb`) is the same loop that `aro ask` uses, in spirit: load the model, build a system prompt from the ARO knowledge base, pass user turns through the tokenizer, and stream back a reply. The difference is that `aro ask` does not require Python, does not require MLX, and does not require you to start Jupyter. It is the same brain in a simpler body.

2.6 A Note on Iteration

One of the later notebooks in the pipeline is `20_iterative_loop.ipynb`. It does something that is easy to describe and hard to do well: it uses the *current* fine-tune to generate new training data for the *next* fine-tune. Every cycle, the model is asked to produce ARO programs for a list of prompts; the ones that pass `aro check` are added to the training set; the ones that fail are added to the DPO negative set. The next round of training learns from both.

This is a feedback loop. It is also the point where the story becomes recursive: the model is now helping train its own successor. Not autonomously — a human still runs the notebooks, reviews the outputs, and decides what to keep. But the leverage is enormous. A morning of curation turns into a weekend of training, which turns into a model that is better at helping you curate.

The first version of `aro-coder-4bit` was bootstrapped from data generated by a general cloud model. Each subsequent version is bootstrapped from data generated by its predecessor — the teacher model is uploaded to HuggingFace after each training cycle, and the next cycle downloads it as its starting point instead of starting from vanilla Qwen. The cloud model has, for the most part, already dropped out of the loop.

Chapter 3: The Commands

> "There is a difference between a tool you can use and a tool you already know. This chapter is about closing that gap."

3.1 The First Invocation

```
$ aro ask "write a feature set that returns OK for GET /health"
```

That is the entire interface. Everything after `aro ask` is a prompt. The first time you run this, the CLI will notice it has no model cached and offer to download one:

```
Model 'ARO-Lang/aro-coder-4bit' (~4.5 GB) is not installed.
Download from Hugging Face? [y/N]
```

Say yes. The weights land in `~/.cache/aro/models/ARO-Lang/aro-coder-4bit/`. Subsequent invocations find them there and skip the download.

If you would rather put the cache somewhere else — a shared network drive, an external SSD — set the `HF_HOME` environment variable before running `aro ask`. If the model is gated and you have a token, set `HF_TOKEN`. Neither is required for the default model.

3.2 Backends

`aro ask` does not ship its own inference engine. It detects and uses whichever of the following it finds first, in order:

1. **Any OpenAI-compatible endpoint** specified by `ARO_ASK_ENDPOINT`. This is how you point `aro ask` at a shared inference server — an office GPU box, a colleague's machine, a dev container. Set `ARO_ASK_API_KEY` if the endpoint needs one.
2. **Native MLX** (macOS Apple Silicon only). In-process inference — no external server needed. This is the preferred backend on Apple Silicon and the fastest option.
3. **`llama-server`** from `llama.cpp`. Install with your package manager or let `aro ask` download it automatically. This is the preferred backend on Linux.
4. **`mlx_lm.server`** from `mlx-lm`. A Python-based fallback on macOS if native MLX is not available.

If none of these are available the command fails with a clear error, not a cryptic one. There is no "automatic fallback to cloud" — the whole point of the local model is that it is local. You choose when to involve anyone else's machine.

Run `aro ask /model` to see which backend `aro ask` picked and where the weights live.

3.3 One-shot vs. REPL

`aro ask` runs in two modes. If you pass a prompt as arguments, it runs one-shot: send the prompt, run tools, print the reply, save the context, exit.

```
$ aro ask "show me how to extract a path parameter"
```

If you run it with no arguments *and* stdin is a terminal, it drops into an interactive REPL.

```
$ aro ask
aro ask - backend: llama.cpp, model: ARO-Lang/aro-coder-4bit
type /quit to exit, /help for commands
lm>
```

The REPL is built on LineNoise, so arrow keys, history, and Ctrl+R search all work the way you expect. Press Ctrl+D or type `/quit` to leave. Type `/help` at any time to see the list of slash commands.

3.4 The Slash Commands

Slash commands are how you talk to `aro ask` itself, instead of to the model. They work in both modes: pass them as the first argument in one-shot, or type them at the REPL prompt.

```
/clean               Delete .context in the current directory.
/show                Print a short summary of the current conversation.
/tools               List every tool the model can call.
/model               Print the active model, its path, and the backend.
/mcp                 List the MCP servers currently bridged into the session.
/index               Walk the project and (re)build the retrieval index.
/search <query>      Debug retrieval: print the top 5 matches for a query.
/fix                 Run aro check, feed diagnostics to the model, auto-repair.
/explain <file>      Ask the model to explain a file or feature set in plain English.
/docs <topic>        Generate documentation for a topic from the current project.
/plugin <name>       Scaffold a new plugin directory with plugin.yaml and stubs.
/openapi <spec>      Generate or update openapi.yaml from a natural-language description.
/quit                Leave the REPL.
```

A few of these are worth a paragraph of their own.

`/clean` deletes the `.context` file in the current directory. Contexts drift over long conversations — the model starts to "remember" things from three tasks ago and applies them to the thing in front of it. When a conversation has clearly lost the plot, `/clean` and start again.

`/tools` is how you discover what the model can actually do. The built-in tools are listed, plus any tool the bridged MCP servers expose. If a colleague ships a new ARO plugin that registers an MCP tool, you'll see it here without any extra configuration on your side.

`/index` walks the project and builds a retrieval index at `.context.index/vectors.json`. Run this once after a fresh clone, and then again any time you move or add a lot of files. Chapter 5 explains what the index is used for.

`/search` is a debugging tool. It shows you which chunks of the project the model would see if it called `search_project` with the same query. Use it to understand why the model did or did not find the thing you expected it to find.

`/fix` is the one you will reach for most often. It works deterministically — no tool-calling loop, no hoping the model picks the right tool. It runs `aro check` directly, reads the source files, builds a focused prompt with the code and the exact error, asks the model for a fix, validates the fix with `aro check` in a temp directory, and writes the corrected files back only if validation passes. It repeats up to five times with decreasing temperature. You can pass a file or a directory: `/fix`

`./MyApp/users.aro` or `/fix ./MyApp`. The workflow it replaces — read error, open file, find line, fix it, re-run check — takes minutes by hand and seconds with `/fix`.

`/explain` is for reading code you did not write. Point it at a file and it produces a paragraph-level explanation of what each feature set does, what events trigger it, and what data flows through it. It is not a substitute for reading the code yourself, but it is a good first pass when you inherit a project.

`/docs` generates documentation from the project in its current state. It reads the relevant `.aro` files, the `openapi.yaml` if one exists, and produces a markdown summary of the application's endpoints, events, and feature sets. Useful for onboarding, and for keeping a wiki page in sync with the code without writing it by hand.

`/plugin` scaffolds a new plugin. Give it a name and it creates the directory under `Plugins/`, writes a `plugin.yaml` with sensible defaults, and stubs out the source files for the plugin type it infers from your environment (Swift on macOS, Rust or C elsewhere). You still have to write the logic, but the boilerplate is done.

`/openapi` is the inverse of the usual workflow. Instead of writing `openapi.yaml` by hand and then writing feature sets to match, you describe your API in plain English — "a user service with CRUD operations and a health check" — and the model generates the OpenAPI spec. It is a starting point, not a final draft. Review it, adjust the schemas, and then let the model write the feature sets to match.

3.5 Flags

`aro ask` has four flags, and you will probably only ever use two of them.

- **`--model <id>`** — override the default model. Only useful if you have trained your own variant.
- **`--yes`** — auto-approve every shell tool call. Use this in scripts and in CI. Never use it when you are about to walk away from the terminal.
- **`--no-mcp`** — skip the MCP bridge bootstrap. Faster startup for slash commands that don't need tools.
- **`--temperature <value>`** — sampling temperature. Defaults to `0.2`, which is deliberately low; the fine-tune was trained to be confident about ARO syntax, and high temperatures make it start inventing verbs again. Raise it only if you want the model to be more creative in non-code explanations.

3.6 The Context File

Every conversation lives in a single file, `.context`, in the current working directory. It is YAML. It is human-readable. You can open it in your editor and read it, you can commit fragments of it into your project if you want to preserve a particularly useful conversation, and you can hand it to a colleague so they can see exactly what you asked and what the model replied.

A shortened example:

```yaml
model: ARO-Lang/aro-coder-4bit
created: 2026-04-06T12:00:00Z
messages:
  - role: system
```

```
    content: "You are ARO-Coder, an assistant specialised in the ARO ..."
  - role: user
    content: "write a feature set that greets a user"
  - role: assistant
    content: |
      ```aro
      (greetUser: User API) {
          Extract the <name> from the <pathParameters: name>.
          Compute the <greeting> from "Hello, " ++ <name> ++ "!".
          Return an <OK: status> with <greeting>.
      }
      ```
```

You can also add an `mcp_servers:` section at the top to bring additional MCP servers into the session:

```
mcp_servers:
  - command: aro
    args: [mcp]
  - command: /opt/my-tools/docs-mcp
    args: [--stdio]
```

The file is written atomically — the CLI writes a `.context.tmp` sibling and renames it into place — and its permissions are restricted to `0600` on Unix systems. It is yours, not the world's.

Chapter 4: Habits

"A sharp tool makes the user faster; a well-habituated tool makes the user calmer."

4.1 Context Is Small

The single most common mistake, made by everyone who picks up `aro ask` for the first time, is treating the context file as a long-lived project log. It is not. It is a scratchpad. The longer it gets, the worse the model gets.

There are two reasons. The first is mechanical: the fine-tune has a context window of 8192 tokens. The longer your `.context` grows, the closer you get to the edge, and the more the model forgets. The second is behavioural: when there are twenty turns of prior conversation in the prompt, the model tends to confuse the current question with a previous one. It starts giving you answers to things you asked an hour ago.

The habit is simple: one task, one `.context`. When you finish a feature, run `/clean`. When you switch from writing a feature to debugging a test, run `/clean`. When you come back to work in the morning and the previous evening's conversation does not match what you want to do now, run `/clean`.

You will not lose anything important. If the conversation was worth keeping, you already committed the code it produced. The chat is not the artifact. The code is the artifact.

4.2 Let It Read the Project

The second most common mistake is pasting code into the prompt. Do not do this. The model has a tool — `read_file` — for precisely this reason. When you want the model to look at `users.aro`, do not paste the file. Say "read users.aro and explain how the validation works." The model will call `read_file` and report back.

The habit generalises. Never describe what a file contains when the model could just read it. Never describe what a directory contains when the model could just `list_dir` it. Never run a command yourself and paste the output when the model could `run_shell` it and get the output directly. The more the model does by itself, the less drift there is between what you think is true and what actually is.

The exceptions are real but small. If a file is enormous, say "read the first hundred lines of …". If the command is destructive, approve it or deny it but do not bypass the approval by running it in another terminal and pasting the result.

4.3 Approve Shell Commands Like You Mean It

When the model wants to run a shell command, `aro ask` stops and asks:

```
[aro ask] approve shell command? [y/N]
  rm -rf build/
>
```

Read the command before typing `y`. Read it *every* time. The fine-tune is good. The fine-tune is not infallible. "Approve-and-forget" is how people end up deleting directories they did not mean

to delete. Typing `y` is muscle memory; pausing for half a second before typing `y` is also muscle memory, and the second one is the muscle memory you want.

The `--yes` flag bypasses the prompt. It exists because there are legitimate, non-interactive reasons to run `aro ask` — CI pipelines, scheduled jobs, batch refactors. Use it in scripts. Do not use it when you are about to leave the terminal unattended. A model that is permitted to run arbitrary shell commands without supervision is, at minimum, an engineer who does not need to sleep. At maximum it is a mistake waiting for a moment of distraction. The safety margin between those two is the `--yes` flag.

4.4 Parse Before You Write

The model has a tool called `parse_aro`. It takes a block of ARO source and returns either `ok` or a parser diagnostic. It does not write anything to disk. It does not touch the filesystem. It just tells you whether what the model is about to write is going to compile.

Train yourself — and the model — to use this tool before writing files. "Parse this first, and if it's clean, write it to `users.aro`." Two calls instead of one, but the second call only happens if the first one succeeds. The result is that your working tree never contains a half-written feature set that does not parse.

4.5 Keep the Index Fresh

The first time you run `aro ask /index` in a project, it walks every indexable file, chunks them, embeds them, and writes the result to `.context.index/vectors.json`. After that, the model has access to a search tool that can find a relevant piece of the project in one call.

The index does not update itself. If you move a file, add a new proposal, or rewrite a large chunk of source, run `/index` again. It is fast. There is no reason not to.

Your `.gitignore` should include `.context.index/`. The index is a cache, not a source of truth; regenerating it is cheap, and committing it would bloat the repo with megabytes of vectors that go stale on every edit. Similarly, decide explicitly whether you want `.context` to be committed. Some teams do — a shared record of the conversations that shaped a feature. Most teams do not. Either is fine; just make the choice deliberately.

4.6 Read the Proposals Before You Argue

When `aro ask` gives you an answer you disagree with, your first instinct will be to argue with it. Your second instinct should be to ask it for a citation. The fine-tune was trained on the `Proposals/` directory, and it has the `read_proposal` tool. Ask: "read ARO-0018 and tell me why you suggested pipeline syntax for this." The model will read the proposal and either reinforce its original answer with a citation, or it will notice it was wrong and correct itself.

The habit is to move disagreements from your head into the spec. Both of you — human and model — get better at ARO this way.

4.7 Commit Messages Are Not the Model's Job

`aro ask` is excellent at writing ARO feature sets, explaining error messages, refactoring handlers, and chasing down compiler diagnostics. It is mediocre at writing commit messages, because the training data did not include many.

If you ask it to write a commit message, you will get one. It will be syntactically fine and semantically vague, because the model does not know what you were trying to accomplish outside of this one conversation. Write the commit message yourself. It will take you thirty seconds and will be better than anything the model produces. This is not a rule. It is a recommendation based on which battles are worth picking.

4.8 Precise Prompts Are Better

For single feature sets, keep prompts short and direct. "Write a feature set for `createUser` that validates the email and emits a `UserCreated` event" is better than "Could you please help me write a feature set that will create a user, and also make sure to validate that the email address they provide is in a valid format, and then after the user is created successfully, it would be good if we could emit an event saying the user was created…"

But when you want a complete application — multiple files, an OpenAPI contract, event handlers, the full structure — give the model a detailed implementation plan. Describe the endpoints, the data models, the event flow, and the file layout. The model was trained on exactly this pattern: read a plan, produce every file. The more precise the plan, the closer the output is to what you want. A good plan names the feature sets, the event types, the repository names, and the API routes. A vague plan gets vague code.

The rule in both cases is the same: say exactly what you mean, no more and no less. For a single feature set, that is one sentence. For a full application, that is a few paragraphs.

Chapter 5: Automation Everyone Understands

> "A script that only its author can read is a liability. A script anyone on the team can read is an asset."

5.1 The Case for Readable Automation

Most automation fails not because it is wrong but because it is illegible. A shell script accumulates flags over the years, each added by someone now on a different team, and eventually nobody is willing to touch it. A Python job grows into three hundred lines of `try` blocks and retry logic, and new joiners are quietly warned not to go near it. The automation works, until it doesn't, and when it doesn't nobody can fix it.

ARO changes the economics of this. Because every ARO feature set is a sequence of English sentences, a person reading an ARO script can understand what it does without reading a single line of Python or shell. Because feature sets are triggered by events, the flow is declarative — you see what happens in what order, not how it happens behind a web of callbacks. And because ARO ships with a local assistant, you can ask the assistant "what does this job do, and what would break it?" and get a real answer.

This chapter is about using `aro ask` to write automation that your non-engineer colleagues can read. Not "should be able to read in principle, with a day of training" — actually read, today, at the same terminal where they already check their email.

5.2 The First Useful Script: A Nightly Report

Start small. Here is a prompt that has produced a working ARO application on every model the fine-tune was evaluated against:

> Write an ARO application that runs once, reads every `.log` file in `./logs`, counts how many lines contain the word "ERROR", writes a report to `./report.md`, and exits.

What `aro ask` will do, if you let it:

1. Call `list_dir` on `./logs` to see what's actually there.
2. Call `parse_aro` on a draft feature set before writing anything.
3. Call `write_file` to create `main.aro`.
4. Call `aro_check` on the resulting directory.
5. Report back with a short explanation and the path to the new file.

You can read the resulting `main.aro`, and so can the person sitting next to you who does not write software. It will look like this:

```
(Application-Start: Nightly Report) {
    Retrieve the <log-files> from the <filesystem>
        where path = "./logs" and extension = ".log".

    For each <file> in <log-files> {
        Retrieve the <content> from the <file>.
        Filter the <error-lines> from the <content>
```

```
            where line contains "ERROR".
        Compute the <error-count: length>
            from the <error-lines>.
        Publish as <counts> append <counts>
            with <error-count>.
    }

    Compute the <total: sum> from the <counts>.
    Compute the <report-text>
        from "Total errors: " ++ <total>.
    Store the <report-text> to the <filesystem>
        where path = "./report.md".

    Return an <OK: status> for the <report>.
}
```

Anyone on the team can read that. Anyone on the team can edit it. The automation is no longer hidden behind someone's personal Python library.

5.3 CI-style Invocations

`aro ask` is scriptable. Here are patterns that have proven themselves:

Ask the assistant to fix a failing check.

```
aro ask --yes \
  "run aro_check on ./Examples/UserService \
   and fix any diagnostics"
```

This will, in order, call `aro_check`, read the offending files, edit them with `edit_file`, re-run the check, and stop when it passes or after 25 rounds of tool calls. The `--yes` flag auto-approves shell access so the loop can run without a human at the keyboard. The result is either a fixed directory or a context file you can open afterwards to see what was tried.

Generate boilerplate for a new operation.

```
aro ask --yes \
  "add a feature set called deleteUser \
   that handles DELETE /users/{id} in ./MyApp"
```

The model will read `openapi.yaml`, extract the schema, write the new `.aro` file, and run `aro_check` to confirm it parses. Your job is to review the diff, not to write the boilerplate.

Run on a shared endpoint.

```
ARO_ASK_ENDPOINT=http://192.168.1.42:8080 \
  aro ask --yes "check everything under Examples/"
```

On CI servers, running `llama-server` per job is wasteful. Run it once, on a machine with a GPU, point every job at it with `ARO_ASK_ENDPOINT`, and `aro ask` will happily use it over the network. The model never leaves your infrastructure, and every job gets a fast response.

Explain changes in a PR.

```
aro ask --no-mcp \
  "read the last five files modified in this \
   branch and write a short PR summary" \
  > pr-summary.md
```

The `--no-mcp` flag skips the MCP bridge boot for a small speedup; the command writes its output to a file you can paste into the PR description. Nothing fancy, just tedium deleted.

5.4 Scheduled Jobs

You can put `aro ask --yes "..."` into a cron job, a systemd timer, or a GitHub Actions schedule. The fact that it runs a local model rather than a cloud service is a feature, not a limitation — your scheduled jobs are not rate-limited, not billed per token, and not waiting on someone else's availability.

Two things to watch for:

- **Approval is off.** With `--yes`, every shell and file tool runs without human review. Make sure the job's working directory is one you're willing to have fully modified.
- **The model can still be wrong.** A scheduled `aro ask` job should produce a diff you review before it lands in production, not commit directly to main. Use the job to propose, not to merge.

A good pattern is to have the job open a pull request rather than push to a branch. The assistant writes the code, the CI runs the tests, and a human approves the merge.

5.5 Talking to Other MCP Servers

Every MCP server you add to the `mcp_servers:` list in your `.context` file becomes a set of tools the model can call during a conversation. This is how you extend `aro ask` without changing the binary.

A few examples of MCP servers worth bridging, if you use them:

- **An internal docs MCP**: the model can read your company wiki without leaking it to a cloud provider.
- **A ticket system MCP**: the model can look up the ticket number in the commit message and pull its description.
- **A secrets broker MCP**: the model can ask "is there a database password configured for `staging`?" without ever seeing the password itself.

The pattern is always the same: add a server to `.context`, run `aro ask /mcp` to confirm it connected, run `aro ask /tools` to see what new tools are available, and then write a prompt that asks the model to use them. The tools appear alongside the built-ins without any special flag. The model does not care whether a tool is built-in or bridged; it just calls it.

5.6 A Principle

The principle threading through this chapter is: automation is readable when the human and the machine are writing in the same language. ARO makes the automation human-readable. `aro ask`

makes the human's job of authoring it fast enough to be worth the investment.

The old argument against domain-specific languages was that they were expensive to write. That argument depended on nobody having an assistant that already knew the language. The assistant is here. The language is here. The only thing left is to sit down and write the first script.

Chapter 6: Reclaiming the Local

> "Every generation of computing begins by promising the user control and ends by renting it back to them. Ours does not have to end that way."

6.1 The Drift to the Cloud

There was a period, roughly from 2020 to 2024, when the trajectory of programming tooling was clear: everything moves to the cloud. The editor, the compiler, the assistant, the test runner, the deployment pipeline — all of it was migrating from the developer's laptop into someone else's data centre. The argument for this was seductive. Cloud services had more compute, newer hardware, bigger models, faster iteration. Your laptop was just a terminal; the real work happened elsewhere.

What went unspoken was that the real work *also cost money*, per token, per CPU-second, per gigabyte of egress. And that the real work could be throttled, or rate-limited, or rewritten on someone else's schedule, or simply switched off. And that the code you fed to the real work was copied, stored, and sometimes used as training data in ways the terms of service made technically legal and morally vague.

This was not a conspiracy. It was the natural economic pull of a market in which compute was expensive and centralisation was efficient. It was also, quietly, a loss. The programmer's laptop — a machine more powerful than any mainframe of 1985 — was reduced to a display for a login screen.

6.2 What Changed

Two things changed, more or less simultaneously.

The first is that inference got dramatically cheaper. A model small enough to run on a laptop with integrated graphics, in 2026, is roughly as capable on code as a flagship cloud model was two years earlier. The capability curve is not flat — the frontier is still in the cloud — but the *useful* line has dropped well below the laptop hardware of ordinary engineers. An 8-billion-parameter coder — 4-bit quantised — fits in memory on machines that cost less than a business-class flight.

The second is that we finally learned how to fine-tune for tasks. A model that knows a little bit about everything is less useful, on a specific domain, than a model that knows only that domain. The ARO fine-tune is a sharp example: it is terrible at explaining kubernetes YAML, because it was never trained on it, but it is excellent at writing ARO feature sets, because that is all it was trained on. A team that knows its domain can build a specialised model that outperforms any general-purpose cloud model *on that domain*, for a fraction of the cost, on a machine they already own.

Put those two changes together and the economic pull reverses. The cloud is no longer the obvious answer for domain-specific programming tasks. The obvious answer is the laptop — *your* laptop, running *your* model, talking to *your* code.

6.3 What `aro ask` Is For

`aro ask` is a small, specific bet on that reversal. It is not a general-purpose coding assistant. It is a coding assistant for one language, running locally, with tools that are scoped to one project. It

does not phone home. It does not require an account. It does not need network access once the model is downloaded. It does not train on your conversations.

That last point is worth dwelling on. Every message you send to `aro ask`, every file it reads, every command it runs, lives on your machine and nowhere else. The `.context` file is yours. The tool-call logs are yours. The history of your thinking, captured in the back-and-forth with the model, is yours. If you want to share a conversation with a colleague, you copy the `.context` file. If you want to delete it, you delete it. There is no "but a copy is still on our servers for thirty days per our retention policy".

This is the most understated feature of the whole tool. It is also, arguably, the most important one.

6.4 The Recursive Game

Chapter 2 ended with a note on the iterative loop notebook — the one where the current version of the fine-tune helps train the next version. That note is worth expanding, because it points at something genuinely new.

A local model that is *good enough* to help improve itself creates a feedback loop that does not require a cloud provider to close. A person with a laptop, a handful of gigabytes of disk space, and a willingness to curate, can train a model that is slightly better at ARO than last week's version. Not in three weeks on a rented GPU cluster. On a laptop, over a weekend. And next week, the slightly-better model helps curate slightly-better training data for the week after. The curve is not steep. It does not need to be. It is locally owned, and it does not stop.

The end state of this game is not "our local model beats GPT-7". It is "our local model is good enough that we no longer need to ask whether a cloud model would be better, because the cost of finding out is more than the value of the answer." That is a different kind of sufficiency than the frontier chases. It is the kind that used to be called "tools the engineer owns".

6.5 What You Can Do Now

If you are reading this and you have an ARO project on your laptop, here is what you can do today, without permission, without a subscription, without an account:

1. Install `aro ask` — it comes with the `aro` binary. You already have it.
2. Install `llama-server` or `mlx_lm.server`. Five minutes.
3. Run `aro ask "hello"`. Answer `y` to the download prompt. Wait.
4. Ask the model to help you write a feature set. Any feature set. The one you have been putting off because it was boring.
5. When it is done, commit the code, delete the `.context`, and go to bed.

That sequence is a small act, but it is an act of claiming something back. The code is on your machine. The model is on your machine. The decision about what to build next is on your machine. There is no cloud to consult, no quota to worry about, no billing email at the end of the month, no vendor's priorities leaking into your architecture.

6.6 Where This Ends

The honest answer is: we do not know where this ends. The fine-tune will get better. The pipeline will get simpler. The hardware will get cheaper. Eventually, someone on the ARO team will train a

variant of the model that specialises in their own application domain — invoicing, logistics, content moderation — and ship it alongside the base `aro-coder`. Someone else will bridge an internal MCP server and get the model to do things we have not thought of yet. Someone else will fork the whole training pipeline and train a model for a language that is *not* ARO, because the pattern generalises.

Every one of those things is a small victory against the drift to the cloud. None of them require anybody's permission.

ARO started with a conversation. So did this chapter of computing. The conversation continues on the laptop in front of you.

The next three chapters shift from philosophy to practice. Chapter 7 explains how the tool-call loop works — the mechanism that turns `aro ask` from a chatbot into a colleague. Chapter 8 collects the practical tips that make the difference between fighting the model and working with it. Chapter 9 walks through complete examples, from an empty directory to a running application, so you can see the whole process from end to end.

Chapter 7: Tool Calling

> "A model that can only talk is a lecturer. A model that can read, write, and run is a colleague."

7.1 The Eighteen Tools

When you type `/tools` in the REPL, you see a list. It looks something like this:

```
Built-in tools:
  read_file         Read a file from the project
  write_file        Write content to a file
  edit_file         Apply a targeted edit to a file
  list_dir          List files and directories
  grep              Search file contents by pattern
  search_project    Semantic search over the project index
  run_shell         Execute a shell command (requires approval)
  parse_aro         Parse ARO source and return diagnostics
  aro_check         Run aro check on a directory
  aro_run           Run aro run on a directory
  aro_test          Run aro test on a directory
  read_proposal     Read an ARO proposal by number
  create_plugin     Scaffold a plugin directory
  write_openapi     Write or update an openapi.yaml file
  write_plugin_yaml Write a plugin.yaml manifest
  explain_error     Explain an ARO diagnostic in plain English
  format_aro        Format ARO source to canonical style
  list_actions      List available ARO actions and their verbs
```

Eighteen tools. Some read. Some write. Some run code. Together they turn the model from something that generates text into something that modifies your project. That distinction matters more than anything else in this book.

A model without tools can suggest a fix. A model with tools can *apply* the fix, check that it compiles, and tell you whether it worked. The difference is the difference between a code review and a pull request.

7.2 How the Loop Works

Every conversation with `aro ask` follows the same loop. It is simple enough to fit in a paragraph.

You send a message. The model reads it and decides what to do. If the answer is obvious, it replies with text. If the answer requires information the model does not have, it generates a *tool call* — a structured request to one of the eighteen tools. `aro ask` executes the tool call, captures the result, and feeds it back to the model. The model reads the result and decides again: reply, or call another tool. This continues until the model has enough information to answer, or until it has made the changes you asked for.

A typical exchange looks like this under the hood:

```
User:     "fix the parser error in users.aro"
Model:    [tool_call: aro_check {"path": "."}]
Runtime:  "users.aro:4: error: unknown verb 'Retreive' - did you mean 'Retrieve'?"
Model:    [tool_call: read_file {"path": "users.aro"}]
Runtime:  (file contents)
Model:    [tool_call: edit_file {"path": "users.aro", "old": "Retreive", "new": "Retrieve"}]
Runtime:  "ok"
Model:    [tool_call: aro_check {"path": "."}]
Runtime:  "all checks passed"
Model:    "Fixed the typo in users.aro line 4: 'Retreive' → 'Retrieve'. Check passes now."
```

Five tool calls. No human intervention between them. The model read the error, found the file, fixed the typo, and verified the fix. This is the tool-call loop at its most routine, and it is the kind of work that `aro ask` does dozens of times a day if you let it.

7.3 The Shape of a Tool Call

A tool call is not a free-form shell command. It is a structured JSON object with a name and parameters. The model cannot call a tool that does not exist. It cannot pass parameters the tool does not accept. The schema of every tool is baked into the system prompt, and the fine-tune was trained to produce calls that match the schema.

This is a deliberate constraint. An unconstrained model that could run arbitrary code would be powerful and terrifying. A constrained model that can only call eighteen well-defined tools is powerful and predictable. You know, at any point, exactly what the model *could* do, because the list of tools is right there in `/tools`.

The constraint also means the model cannot surprise you with tools you did not know about — unless you bridge an MCP server that adds new ones, in which case you chose to add them, and `/tools` will show you what they are.

7.4 Approval and Trust

Not all tools are created equal. `read_file` is harmless — it reads a file and returns its contents. `run_shell` is not harmless — it executes an arbitrary shell command. `aro ask` knows the difference.

Tools are classified into three tiers:

- **Read-only tools** (`read_file`, `list_dir`, `grep`, `search_project`, `read_proposal`, `list_actions`, `explain_error`): these run without asking. The model can read your project freely. This is by design — a model that has to ask permission to read a file is too slow to be useful.
- **Write tools** (`write_file`, `edit_file`, `write_openapi`, `write_plugin_yaml`, `create_plugin`, `format_aro`): these run without asking in REPL mode, but every change is printed to the terminal so you can see what happened. In one-shot mode with `--yes`, they run silently. Without `--yes`, one-shot mode prints the change and asks for confirmation.
- **Execution tools** (`run_shell`, `aro_run`, `aro_check`, `aro_test`, `parse_aro`): `aro_check`, `aro_test`, and `parse_aro` are considered safe and run without asking, because they do not modify anything. `aro_run` and `run_shell` always ask for approval unless `--yes` is set.

The dividing line is simple: tools that cannot change your project or execute arbitrary code run

freely. Tools that can change files print what they did. Tools that can run your code ask first. If you want to move the line — make everything auto-approved — use `--yes`. If you want to move it the other way, there is no flag for that, because the default is already cautious.

7.5 Chaining: Read, Edit, Check

The most common tool-call pattern is the three-step chain: read a file, edit it, and check the result. You will see this pattern over and over, and it is worth understanding why it works so well.

The model does not edit files blind. It reads first. This matters because the edit tool uses exact string matching — the model provides the old text and the new text, and the tool replaces one with the other. If the model has not read the file, it cannot know what the old text looks like, and the edit will fail. The fine-tune learned this the hard way during training, and now it almost always reads before editing.

After the edit, the model calls `aro_check` or `parse_aro` to verify that the change did not break anything. If the check fails, it reads the new diagnostic, edits again, and checks again. This loop — edit, check, edit, check — runs until the check passes or the model decides it cannot fix the problem and asks you for help.

The pattern generalises. For creating a new file:

```
write_openapi → write_file (main.aro) → aro_check → edit_file (fix) → aro_check
```

For adding a feature to an existing application:

```
list_dir → read_file (openapi.yaml) → read_file (main.aro) → write_file (new.aro) → aro_check
```

For debugging a runtime error:

```
aro_run → read error → read_file → edit_file → aro_run → confirm fix
```

Every chain follows the same rhythm: gather information, make a change, verify the change. If you watch the tool calls scroll by in the terminal, you will start to see the rhythm, and you will start to trust it — or to interrupt it when something looks wrong.

7.6 Creating a Project from Scratch

Here is what happens when you ask `aro ask` to build something from nothing. Say you type:

```
"create an ARO application called HealthCheck with a single GET /health endpoint that returns
```

The model will typically make these calls, in this order:

1. `list_dir` on the current directory to see what exists.
2. `write_openapi` to create `HealthCheck/openapi.yaml` with a single path.
3. `write_file` to create `HealthCheck/main.aro` with `Application-Start` and the `healthCheck` feature set.
4. `aro_check` on `HealthCheck/` to verify everything parses.
5. If the check fails, `read_file` and `edit_file` to fix whatever it got wrong.
6. A final `aro_check` to confirm.

The result is a directory you can immediately run with `aro run ./HealthCheck`. Two files, both correct, both checked. The model did not guess at the directory structure or the OpenAPI format — it used dedicated tools that know the conventions.

7.7 The Limits of Tools

Tools make the model useful. They do not make the model infallible.

The model can call the wrong tool. It occasionally calls `write_file` when it should call `edit_file`, overwriting content it meant to preserve. It sometimes runs `aro_check` on the wrong directory. It sometimes calls `run_shell` with a command that makes sense on Linux but not on macOS, or the other way around.

The model can also get stuck in a loop. If a check keeps failing and the model keeps making the same edit, it will cycle until the tool-call limit is reached (25 rounds by default). When this happens, the model gives up and reports what it tried. This is the right behaviour — an infinite loop of bad edits would be worse — but it means you sometimes have to step in and fix the problem yourself.

The defence against both failure modes is the same: watch the tool calls. They are printed to the terminal as they happen. If you see the model heading in the wrong direction, press Ctrl+C. The conversation is saved, and you can steer it back on course with your next message.

7.8 Tips for Better Tool Usage

A few patterns reliably get better results from the tool-call loop.

Be specific about paths. "Fix the error in users.aro" is better than "fix the error". The model will find the file either way, but the specific prompt saves a `list_dir` and a `grep`, which saves time and context window.

Ask for verification. "Write the feature set and run aro check" is better than "write the feature set". The model often checks on its own, but explicitly asking makes it reliable.

One task per conversation. The model is better at tool-calling when the goal is clear. "Add a deleteUser endpoint" is one task. "Add deleteUser, refactor the repository pattern, and update the README" is three tasks, and the model will try to do all three in one chain of tool calls, which gets messy. Do them one at a time. Use `/clean` between them.

Let it fail. If the first edit does not fix the check, do not interrupt. Let the model see the failure and try again. The second attempt is usually better than the first, because the model now has the diagnostic *and* the knowledge of what did not work. Interrupting after the first failure robs it of that learning.

Read the diffs. Every `edit_file` call prints the change. Read it. Not because the model is untrustworthy, but because reading the diff is faster than reading the whole file, and it keeps you in the loop. The model is a colleague, not a contractor. You are still responsible for the code.

Chapter 8: Tips and Tricks

"Mastery is not knowing the tool's features. It is knowing which features to use when."

8.1 Writing Good Prompts

The fine-tune responds to shape. It was trained on thousands of instruction/response pairs, and every instruction had the same structure: a verb, a subject, and constraints. The closer your prompt matches that structure, the better the response.

Good prompts:

- "Write a feature set for `createOrder` that validates the total is positive and emits an `OrderCreated` event."
- "Add a `deleteUser` endpoint to the OpenAPI spec and write the matching feature set."
- "Explain why this feature set uses `Publish as` instead of returning the value directly."

Bad prompts:

- "I'm trying to build a thing where users can create orders and I want to make sure the order total is a positive number and also after the order is created I want to send out some kind of event so other parts of the system know about it…"
- "Can you help me?"
- "Write me a complete e-commerce application."

The pattern is: tell the model what to do, tell it what constraints matter, and stop. Do not explain your reasoning. Do not provide background the model did not ask for. Do not hedge. The model is not a person who needs to be convinced. It is a tool that needs a clear instruction.

There is one exception. When you are asking the model to make a judgement call — "should this be one feature set or two?" — a sentence of context helps. "This endpoint returns paginated results and also sets a cache header; should that be one feature set or two?" The context narrows the judgement space.

8.2 Common Mistakes the Model Makes

The fine-tune is good. It is not perfect. Here are the mistakes you will see most often, and what to do about them.

Inventing verbs. The model occasionally generates a verb that does not exist in ARO. "Process the order" sounds natural, but `Process` is not a registered action verb. When this happens, `aro check` will catch it. The fix is usually obvious: `Process` should be `Compute` or `Transform`. You can tell the model "Process is not a valid verb, use Compute instead" and it will correct itself.

Wrong prepositions. Each action verb takes specific prepositions. `Retrieve ... from` is correct. `Retrieve ... with` is not. The fine-tune mostly gets this right, but under long contexts it drifts. Run `aro check` early and often — it validates prepositions.

Forgetting angle brackets. ARO uses `<angle-brackets>` for all identifiers in statements. The model sometimes drops them, especially when it is explaining code in prose and then switches to writing code. A missing bracket is a parse error. `aro check` catches it. `/fix` fixes it.

Overcomplicating. ARO's error philosophy is "code is the error message". The happy case is all you write. The model sometimes forgets this and adds `When` guards for error conditions that the runtime handles automatically. If you see a feature set that is twice as long as you expected, look for defensive guards and remove them. Or tell the model: "remove the error handling, ARO handles that at runtime."

Confusing feature set triggers. The model sometimes writes a feature set named `createUser` and assigns it to a business activity that does not match the OpenAPI operationId. The feature set will parse but will never be triggered. Check that the name in parentheses matches the operationId in `openapi.yaml`.

8.3 Using /fix vs. Manual Editing

`/fix` is the fastest path from "broken" to "working". It runs the check, reads the diagnostics, and applies edits. For simple errors — typos, missing brackets, wrong prepositions — it is nearly always right on the first try.

But `/fix` is not always the right choice. Here is the dividing line.

Use `/fix` when: - The error is syntactic. A misspelled verb, a missing period, a malformed feature set header. - The error is local. One file, one line, one obvious fix. - You want speed. `/fix` is faster than opening the file, finding the line, and editing it yourself.

Edit manually when: - The error is semantic. The code parses but does the wrong thing. - The fix requires understanding context that the model does not have. A business rule that is not written down anywhere, a naming convention your team uses, a dependency between files. - You are learning. Reading the error and fixing it yourself teaches you ARO. Letting the model fix it teaches you nothing.

A good rhythm is: let `/fix` handle the mechanical errors, and handle the design errors yourself. Over time, the ratio shifts — you make fewer mechanical errors, and the model handles more of the design, because you have trained yourself to write better prompts and the model has been fine-tuned to understand your patterns.

8.4 Building Incrementally

The best way to build an ARO application with `aro ask` is one feature set at a time. Not because the model cannot write ten feature sets in one go — it can — but because you cannot review ten feature sets in one go. Not well.

The workflow:

1. Write or generate `openapi.yaml` with `/openapi`.
2. Ask the model to write the `Application-Start` feature set. Review it. Run `aro check`.
3. Pick the next endpoint. Ask the model to write the feature set. Review it. Run `aro check`.
4. Repeat until the application is complete.
5. Run `aro run` and test it.

Each step is a single conversation turn. Each step produces a single file. Each step ends with a check that passes. At no point do you have a project with five unreviewed files and a check that fails for reasons you cannot untangle.

The incremental approach also means you can `/clean` between steps without losing anything. The code is committed. The context is disposable.

8.5 Using .context Effectively

The `.context` file is your conversation state. It grows with every turn, and as it grows, the model's effective context window shrinks. On the default 8192-token window, a long conversation can push the system prompt and the current question off the edge, and the model starts giving answers that ignore the project structure it was told about three turns ago.

The rule is: `/clean` more often than you think you should.

Some specific signals that it is time to `/clean`:

- You have switched tasks. The context from the previous task is noise.
- The model repeats itself or gives an answer that contradicts what it said two turns ago.
- The model stops calling tools and starts guessing at file contents it could just read.
- You have been going back and forth for more than ten turns on the same problem.

After `/clean`, the model starts fresh. It re-reads the system prompt, re-discovers the project, and gives answers that are based on what is actually there, not on what it remembers from an earlier turn. This feels wasteful. It is not. A fresh context with one good question produces better results than a stale context with ten turns of accumulated confusion.

8.6 Combining aro ask with the CLI

`aro ask` is one command in the `aro` toolchain. It works best when you use it alongside the others, not instead of them.

`aro check` validates syntax without running anything. Use it after every edit, whether the edit came from you or from the model. The model calls it internally via the `aro_check` tool, but running it yourself in another terminal gives you a second pair of eyes.

`aro run` executes the application. The model can call this via `aro_run`, but you should run it yourself when you want to see the actual HTTP responses, the actual log output, the actual behaviour. The model sees the stdout; you see whether the application does what your users need.

`aro test` runs colocated tests. If your project has test feature sets, run them after every change. The model can run them via `aro_test`, but the habit of running tests yourself — not just having the model run them — is what keeps you honest.

`aro build` compiles to a native binary. The model does not have a tool for this, because building is a deployment decision, not a development one. Build when you are ready to ship, not when the model tells you to.

The pattern is: use `aro ask` for generation and debugging, use the other commands for verification and deployment. The model is good at writing code and finding bugs. You are good at deciding whether the code does the right thing.

8.7 Performance Tips

The fine-tune runs locally, and local inference has constraints that cloud inference does not. A few things make a noticeable difference.

Temperature. The default is `0.2`. Leave it there. Higher temperatures produce more creative prose but less accurate ARO. If you are asking for an explanation, `0.4` is fine. If you are asking for code, `0.2` or lower. The training data was terse and correct; the model performs best when it is not being asked to improvise.

Model selection. The default `aro-coder-4bit` is a 4-bit quantised 8-billion-parameter dense model — about 4.5 GB on disk. It runs comfortably on any machine with 8 GB of memory. If tokens come out slowly or the fan spins up, check that you are using the right backend for your hardware (see section 3.2).

Backend. On Apple Silicon, the native MLX backend runs in-process and is the fastest option. On Linux with an NVIDIA GPU, `llama-server` with CUDA is the way to go. On CPU-only machines, both are slow, and the best optimisation is to point `ARO_ASK_ENDPOINT` at a machine that has a GPU.

Context length. Keep conversations short. The model's context window is 8192 tokens. A long conversation does not just degrade quality — it also slows inference, because every token of context has to be processed on every turn. `/clean` is a performance optimisation as much as a quality one.

Indexing. Run `/index` once after cloning a project. The retrieval index lets the model find relevant files without scanning the whole directory tree, which saves tool calls and time. A project with an index gets better answers faster than one without.

Chapter 9: Worked Examples

"Theory is what you know before you try it. Practice is what you know after."

9.1 Example 1: A User Service from Scratch

This example builds a complete REST API for managing users — list, create, get, delete — using nothing but `aro ask` in a fresh directory.

The Opening Prompt

```
$ mkdir UserService && cd UserService
$ aro ask
lm> create an openapi.yaml for a user service with CRUD operations on /users and /users/{id}
```

The model calls `write_openapi` and produces:

```yaml
openapi: 3.0.3
info:
  title: User Service
  version: 1.0.0
paths:
  /users:
    get:
      operationId: listUsers
      summary: List all users
      responses:
        '200':
          description: OK
    post:
      operationId: createUser
      summary: Create a user
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              properties:
                name:
                  type: string
                email:
                  type: string
      responses:
        '201':
          description: Created
  /users/{id}:
    get:
```

```
    operationId: getUser
    summary: Get a user by ID
    parameters:
      - name: id
        in: path
        required: true
        schema:
          type: string
    responses:
      '200':
        description: OK
  delete:
    operationId: deleteUser
    summary: Delete a user
    parameters:
      - name: id
        in: path
        required: true
        schema:
          type: string
    responses:
      '204':
        description: Deleted
```

The model then says it has written `openapi.yaml`. The tool call log shows a single `write_openapi` call. No edits needed.

The Application Start

```
lm> write main.aro with Application-Start that starts the HTTP server
```

The model calls `write_file` with path `main.aro`:

```
(Application-Start: User Service) {
    Log "User Service starting..." to the <console>.
    Start the <http-server> with <contract>.
    Keepalive the <application> for the <events>.
    Return an <OK: status> for the <startup>.
}
```

Then it calls `aro_check` on the directory. The check passes.

The Feature Sets

```
lm> write users.aro with feature sets for all four operations in the OpenAPI spec
```

The model calls `read_file` on `openapi.yaml` to refresh its memory of the operationIds, then calls `write_file` with path `users.aro`:

```
(listUsers: User Service) {
    Retrieve the <users> from the <user-repository>.
    Return an <OK: status> with <users>.
}

(createUser: User Service) {
    Extract the <data> from the <request: body>.
    Create the <user> with <data>.
    Store the <user> to the <user-repository>.
    Emit a <UserCreated: event> with <user>.
    Return a <Created: status> with <user>.
}

(getUser: User Service) {
    Extract the <id> from the <pathParameters: id>.
    Retrieve the <user> from the <user-repository> where id = <id>.
    Return an <OK: status> with <user>.
}

(deleteUser: User Service) {
    Extract the <id> from the <pathParameters: id>.
    Retrieve the <user> from the <user-repository> where id = <id>.
    Delete the <user> from the <user-repository> where id = <id>.
    Emit a <UserDeleted: event> with <user>.
    Return a <NoContent: status> for the <deletion>.
}
```

The model calls `aro_check`. The check passes. Four feature sets, four endpoints, all matching the OpenAPI contract.

Running It

```
lm> /quit
$ aro run .
User Service starting...
HTTP server listening on 0.0.0.0:8080
```

In another terminal:

```
$ curl -s localhost:8080/users
[]
$ curl -s -X POST localhost:8080/users -H 'Content-Type: application/json' -d '{"name":"Ada","e
{"id":"1","name":"Ada","email":"ada@example.com"}
$ curl -s localhost:8080/users
[{"id":"1","name":"Ada","email":"ada@example.com"}]
$ curl -s localhost:8080/users/1
{"id":"1","name":"Ada","email":"ada@example.com"}
```

Three files. Two minutes. A working REST API.

9.2 Example 2: A File Watcher

This example builds an application that watches a directory for changes and logs them.

The Prompt

```
$ mkdir FileWatcher && cd FileWatcher
$ aro ask "create an ARO application that watches the current directory for file changes and lo
```

This time we use one-shot mode. The model makes several tool calls in sequence:

1. `list_dir` on `.` — empty directory, nothing to conflict with.
2. `write_file` for `main.aro`:

```
(Application-Start: File Watcher) {
    Log "File watcher starting..." to the <console>.
    Start the <file-monitor> with ".".
    Keepalive the <application> for the <events>.
    Return an <OK: status> for the <startup>.
}

(File Changed: File Event Handler) {
    Extract the <path> from the <event: path>.
    Extract the <kind> from the <event: kind>.
    Compute the <message> from "[" ++ <kind> ++ "] " ++ <path>.
    Log <message> to the <console>.
    Return an <OK: status> for the <notification>.
}
```

3. `aro_check` on `.` — passes.

The model replies: "Created `main.aro` with two feature sets. `Application-Start` starts a file monitor on the current directory and keeps the application alive. `File Changed` handles file events by extracting the path and kind and logging them. Run with `aro run .`."

Testing It

```
$ aro run .
File watcher starting...
```

In another terminal:

```
$ echo "hello" > test.txt
$ echo "world" >> test.txt
$ rm test.txt
```

Back in the first terminal:

```
[created] ./test.txt
[modified] ./test.txt
```

```
[deleted] ./test.txt
```

One file. One prompt. A working file watcher.

9.3 Example 3: A Plugin from Scratch

This example builds a Swift plugin that provides a `slug` qualifier — converting a string like "Hello World" into "hello-world".

Scaffolding

```
$ cd MyApp
$ aro ask
lm> /plugin slug-generator
```

The model calls `create_plugin` with the name `slug-generator`. This creates:

```
Plugins/
   slug-generator/
       plugin.yaml
       Sources/
           SlugGenerator.swift
```

The model reports the scaffold is ready.

Defining the Plugin

```
lm> update the plugin.yaml to provide a slug qualifier with handle SlugGenerator, then impleme
```

The model calls `edit_file` on `Plugins/slug-generator/plugin.yaml`:

```yaml
name: slug-generator
version: 1.0.0
handle: SlugGenerator
provides:
  - type: swift-plugin
    path: Sources/
```

Then it calls `read_file` on `Sources/SlugGenerator.swift` to see the stub, and `edit_file` to replace it:

```swift
import Foundation

@_cdecl("aro_plugin_info")
public func pluginInfo() -> UnsafeMutablePointer<CChar> {
    let info = """
    {
        "name": "slug-generator",
        "version": "1.0.0",
        "qualifiers": ["slug"]
    }
```

```
    """
    return strdup(info)
}

@_cdecl("aro_plugin_qualifier")
public func pluginQualifier(
    _ qualifier: UnsafePointer<CChar>,
    _ inputJSON: UnsafePointer<CChar>
) -> UnsafeMutablePointer<CChar> {
    let qual = String(cString: qualifier)
    let input = String(cString: inputJSON)

    guard qual == "slug" else {
        return strdup("{\"error\": \"unknown qualifier\"}")
    }

    // Parse the input JSON to extract the value
    let value = input
        .replacingOccurrences(of: "{\"value\":\"", with: "")
        .replacingOccurrences(of: "\"}", with: "")

    let slug = value
        .lowercased()
        .replacingOccurrences(of: " ", with: "-")
        .filter { $0.isLetter || $0.isNumber || $0 == "-" }

    return strdup("{\"value\": \"\(slug)\"}")
}

@_cdecl("aro_plugin_execute")
public func pluginExecute(
    _ action: UnsafePointer<CChar>,
    _ inputJSON: UnsafePointer<CChar>
) -> UnsafeMutablePointer<CChar> {
    return strdup("{\"error\": \"no actions provided\"}")
}

@_cdecl("aro_plugin_free")
public func pluginFree(_ ptr: UnsafeMutablePointer<CChar>) {
    free(ptr)
}
```

The model calls `aro_check` on the project root. Passes.

Using the Plugin

```
lm> write a feature set that uses the slug qualifier on a title extracted from the request bod
```

The model calls `write_file` to create `slugify.aro`:

```
(createPost: Blog API) {
    Extract the <title> from the <request: body>.
    Compute the <slug: SlugGenerator.slug> from the <title>.
    Compute the <post> from <title> and <slug>.
    Store the <post> to the <post-repository>.
    Return a <Created: status> with <post>.
}
```

The model calls `aro_check`. Passes. The qualifier is referenced as `SlugGenerator.slug` — the handle from `plugin.yaml` dot the qualifier name from `aro_plugin_info`.

The Full Picture

Three prompts. A plugin directory with a manifest and source. A feature set that uses the plugin's qualifier. Everything checked, everything parseable, everything following the conventions documented in the proposals.

The model did not memorise the C ABI for ARO plugins. It was trained on the proposals and the examples in the `Examples/` directory, and it applied that knowledge through its tools. The `create_plugin` tool gave it the scaffold. The `read_file` tool let it see the stub. The `edit_file` tool let it fill in the implementation. The `aro_check` tool confirmed it worked.

That is the tool-call loop doing what it was designed to do: turning a description of what you want into a project that works, one verified step at a time.

9.4 What the Examples Show

All three examples follow the same arc. You describe what you want. The model reads the project, writes files, and checks its work. You review the result and run it. The conversation is short — three to five turns — because the model has tools that let it act instead of explain.

The examples also show what the model does *not* do. It does not write tests unless you ask. It does not set up deployment. It does not make architectural decisions about things you did not mention. It stays in its lane: ARO code, ARO tooling, ARO conventions. Everything else is yours.

That division of labour is the point. The model handles the syntax, the boilerplate, the mechanical correctness. You handle the design, the naming, the business logic that only you know. Between the two of you, a working application emerges faster than either of you could produce alone.

Appendix A: The Training Pipeline

"The pipeline is twenty-four notebooks, a config file, and a great deal of patience."

This appendix is for people who want to run the training pipeline themselves, extend it, or understand what each step does at a technical level. If you only want to use `aro ask`, you do not need any of this. Chapter 2 covers what the model learned and why; this appendix covers *how*.

A.1 Prerequisites

The pipeline runs on Apple Silicon (M1 or later) with at least 16 GB of unified memory. It uses MLX for local inference and fine-tuning. You will need:

- Python 3.12+ with `mlx-lm`, `matplotlib`, `transformers`
- The `aro` binary on your PATH (for `aro check` and `aro run` validation)
- A HuggingFace account and `huggingface-cli login` (for model upload)

The pipeline lives in `ARO-Train/Train/script/`. All notebooks share a common configuration through `config.py`.

A.2 The Notebooks

Data Collection (NB00-NB02)

NB00 (init) sets up the directory structure and configuration.

NB01 (corpus collection) indexes every source of truth: the 65 Examples, the Language Guide, the Book, the Proposals, the Wiki, and the runtime's Swift source code for action metadata. Output: a manifest of ~400 items.

NB02 (knowledge extraction) parses the manifest into structured knowledge: 54 actions with their verbs and prepositions, 115 examples with their source code, 61 proposals with Q&A seeds. Output: `knowledge.json`.

Training Pair Generation (NB03-NB13)

NB03 (LLM knowledge extraction) is the first notebook that calls the model. For each real example, book chapter, and proposal, it generates instruction/response pairs. It validates every generated code block with `aro check`, and when validation fails, feeds the error back to the model for up to two repair attempts. Bare code snippets from proposals are auto-wrapped in feature sets before checking. Output: ~700 pairs.

NB04 (warm-start fine-tune) trains the base model on all pairs collected so far. This gives the model enough ARO knowledge to generate useful code in later notebooks. The adapter is saved and loaded by every subsequent notebook. Training runs for two full epochs with batch size 4.

NB05 (actions training) generates pairs for every ARO action: usage examples, alias mappings, explanations, "which action" questions, and in-context feature sets. Also includes 16 static error-pattern pairs covering common mistakes (++ vs +, reserved prefixes, wrong prepositions). Output: ~350 pairs across 59 verbs.

NB06 (execution-grounded pairs) generates code that must actually *run*, not just parse. Four strategies: mutation of existing examples, recombination of two examples into one, spec-to-code from proposals, and readme-to-code from example descriptions. Every pair is validated with both `aro check` and `aro run`. Output: ~300 pairs.

NB07 (book Q&A) extracts question/answer pairs from the Language Guide chapters.

NB08 (wiki training) mines the project wiki for training pairs.

NB09 (git training) mines real fix and refactor commits from the git history of ARO applications. A sanitisation step removes old-style `<Verb>` syntax and validates all code blocks with `aro check`.

NB10 (synthetic data generation) is the largest notebook. It generates 3,500+ samples across seven task types: code generation, debugging, correction, full application, fill-in-the-middle, syntax Q&A, and code explanation. It also generates multi-feature-set "architecture" applications. A self-repair loop with targeted error hints fixes common syntax mistakes.

NB11 (function calling) teaches the model to invoke `aro ask` tools correctly. Training pairs cover direct tool calls (with correct JSON arguments), the `/fix` workflow chain (read → check → edit → verify), and tool selection for common tasks.

NB12-NB13 (application prompts, external repos) generate additional training pairs from application plan files and external ARO repositories.

Validation & Assembly (NB14-NB17)

NB14-NB15 (comment extraction, validation) extract comments from existing code and validate all collected pairs.

NB16 (dataset assembly) combines all pairs into a single training dataset with train/validation/test splits.

NB17 (fine-tune) runs the full LoRA fine-tune on the assembled dataset.

Optimisation & Evaluation (NB18-NB20)

NB18 (DPO) runs Direct Preference Optimisation using chosen/rejected pairs built from `aro check` validation.

NB19 (evaluation) evaluates the model against a fixed set of probe prompts and reports syntax pass rates.

NB20 (iterative loop) uses the current model to generate new training data, validates it, adds passing samples to the training set and failing samples to the DPO negatives, then retrains. Multiple rounds.

Distillation & Release (NB21-NB24)

NB21 (distillation) distils the 30B teacher model into an 8B student. The teacher generates 5,000 validated outputs; the student is fine-tuned on the merged dataset.

NB22 (package) quantises the best model to 4-bit, generates a README, smoke-tests the output, and uploads both the distilled student (`ARO-Lang/aro-coder-4bit`) and the teacher (`ARO-Lang/aro-teacher-30b-4bit`) to HuggingFace.

NB23 (chat) is a live test environment for the packaged model.

A.3 The Model Lifecycle

The lifecycle flows top-to-bottom:

1. **Base model** (Qwen 30B or previous teacher from HF)
2. **Warm-start** (NB04: LoRA on ~4,000 pairs)
3. **Full fine-tune** (NB17: LoRA on full dataset)
4. **DPO** (NB18: preference optimisation)
5. **Iterative loop** (NB20: generate, validate, retrain)
6. **Upload teacher** to `ARO-Lang/aro-teacher-30b-4bit`
7. **Distil** 30B teacher into 8B student (NB21)
8. **Upload student** to `ARO-Lang/aro-coder-4bit`
9. **End user** downloads `aro-coder-4bit` via `aro ask`

A.4 Iterative Improvement

After the first complete pipeline run, set `TRAIN_ON_BASE = False` in `config.py`. On the next run, the pipeline will download the teacher model from HuggingFace instead of starting from vanilla Qwen. Each cycle builds on the previous one.

The teacher model (`ARO-Lang/aro-teacher-30b-4bit`) is the full 30B model after all fine-tuning and DPO. The student model (`ARO-Lang/aro-coder-4bit`) is the distilled 8B version for everyday inference. Both are uploaded after each training cycle.

A.5 Key Configuration

`TRAIN_ON_BASE` `True` for fresh start, `False` to resume from the HF teacher.
`MODEL_ID` Resolved at import time. Used by all notebooks.
`TEACHER_MODEL_ID` `ARO-Lang/aro-teacher-30b-4bit`
`PREFERRED_MODEL_ID` `ARO-Lang/aro-coder-4bit` (distilled student)
`STUDENT_MODEL_ID` `mlx-community/Qwen3-8B-4bit`
`BASE_MODEL_ID` Vanilla Qwen 30B MoE (fallback)

All configuration lives in `Train/script/config.py`.